OBJEKTORIENTIERTE PROGRAMMIERUNG (OOP) • PRÄSENTATIONS-HANDOUT

Vererbung (Inheritance) in Python

Teil 1: Grundkonzept & Architektur • Teil 2: Praxis-Code • Teil 3: Python-Besonderheiten

TEIL 1 Was ist Vererbung? (Allgemeines Konzept)

Die **Vererbung** ist eines der vier fundamentalen Grundprinzipien der objektorientierten Programmierung (neben Kapselung, Abstraktion und Polymorphismus). Sie ermöglicht es, eine neue Klasse auf Basis einer bereits existierenden Klasse zu definieren. Dabei übernimmt die abgeleitete Klasse automatisch sämtliche Attribute (Daten) und Methoden (Funktionen) der ursprünglichen Klasse.

Die „Ist-ein"-Beziehung (Is-A Relationship) & das DRY-Prinzip

Vererbung modelliert eine logische Hierarchie vom Allgemeinen zum Speziellen: Ein *Hund ist ein Tier*, ein *PKW ist ein Fahrzeug*. Gemeinsamer Code wird nur einmal in der Basisklasse geschrieben (**DRY** – *Don't Repeat Yourself*). Das reduziert Redundanzen, vermeidet Copy-Paste-Fehler und erleichtert spätere Anpassungen erheblich.

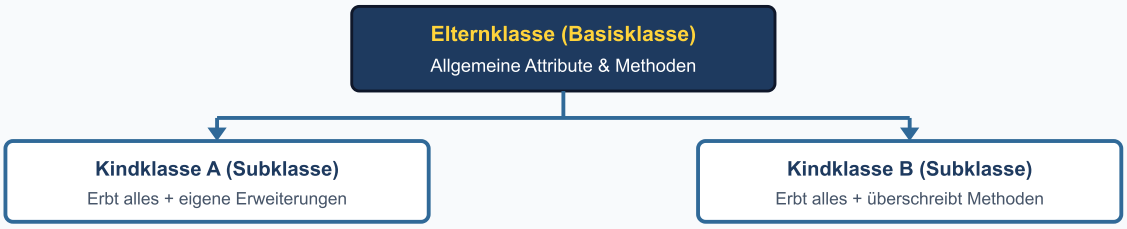


Abb. 1: Schematischer Aufbau einer Klassenhierarchie – Spezialisierung nach unten.

BEGRIFFE Die vier zentralen Bausteine der Vererbung

- **Elternklasse (Basisklasse / Superklasse):** Die übergeordnete Klasse, die das allgemeine Grundgerüst (gemeinsame Eigenschaften und Standardverhalten) für alle abgeleiteten Klassen definiert.
- **Kindklasse (Subklasse / Abgeleitete Klasse):** Die spezialisierte Klasse, die von der Elternklasse erbt. Sie übernimmt deren Code ohne Neuschreiben und ergänzt spezifische Merkmale.
- **Erweiterung (Extension):** Eine Kindklasse fügt komplett neue Attribute oder Methoden hinzu, über die die allgemeine Elternklasse noch nicht verfügt.
- **Überschreiben (Method Overriding):** Eine Kindklasse definiert eine geerbte Methode unter demselben Namen neu, um das Standardverhalten spezifisch für sich anzupassen.

TEIL 2 Praxis-Beispiel in Python

In Python wird Vererbung syntaktisch direkt bei der Klassendefinition angegeben, indem der Name der Elternklasse in runden Klammern hinter den Namen der Kindklasse geschrieben wird: `class Kind(Eltern):`.

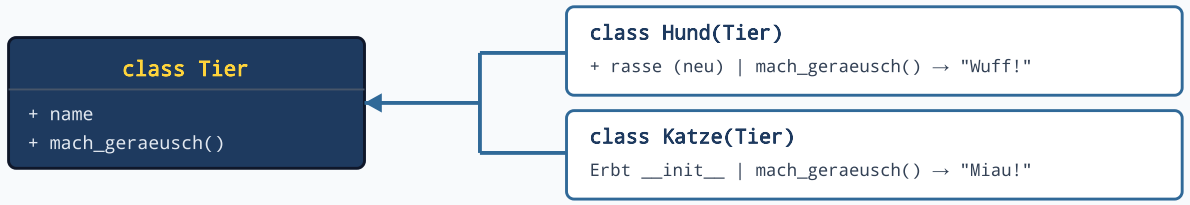


Abb. 2: Die Basisklasse „Tier“ gibt ihre Struktur an „Hund“ und „Katze“ weiter.

```
# 1. Die Elternklasse (Basisklasse) class Tier: def __init__(self, name): self.name = name def mach_geraeusch(self): return "Ein unbekanntes Geräusch" # 2. Kindklasse 'Hund' erbt von 'Tier' und erweitert den Konstruktor class Hund(Tier): def __init__(self, name, rasse): super().__init__(name) # Ruft __init__ der Elternklasse auf self.rasse = rasse # Neues, eigenes Attribut hinzufügen def mach_geraeusch(self): # Überschreibt Methode der Elternklasse return "Wuff!" # 3. Kindklasse 'Katze' übernimmt __init__ direkt von 'Tier' class Katze(Tier): def mach_geraeusch(self): return "Miau!" # --- Ausführung des Skripts --- mein_hund = Hund("Bello", "Labrador") meine_katze = Katze("Minka") print(f"{mein_hund.name} ({mein_hund.rasse}) macht: {mein_hund.mach_geraeusch()}") print(f"{meine_katze.name} macht: {meine_katze.mach_geraeusch()}")
```

KONSOLENAUSGABE (OUTPUT)

```
Bello (Labrador) macht: Wuff!
Minka macht: Miau!
```

- **Wiederverwendung mit `super()`:** Da `Hund` einen eigenen Konstruktor (`__init__`) für das Attribut `rasse` definiert, würde der Konstruktor von `Tier` überschrieben. Mit `super().__init__(name)` delegiert `Hund` das Setzen des Namens sauber an die Elternklasse.
- **Automatische Übernahme:** Die Klasse `Katze` benötigt keine neuen Attribute. Sie verzichtet auf ein eigenes `__init__` und nutzt automatisch den Konstruktor von `Tier`.

TEIL 3 Besonderheiten der Vererbung in Python

Obwohl das Grundkonzept der Vererbung in fast allen objektorientierten Sprachen existiert, verfolgt Python eine besonders dynamische und offene Philosophie. Die folgenden fünf Merkmale unterscheiden Python deutlich von strikt typisierten Sprachen wie Java oder C#:

Merkmal	Klassisches OOP (z. B. Java / C#)	Python-Besonderheit
Mehrfachvererbung	Klassen erben nur von *einer* Basisklasse (Kombinierbarkeit nur über Interfaces).	Direkte Mehrfachvererbung aus beliebig vielen Klassen möglich: `class C(A, B):`.
Suchreihenfolge	Einfache lineare Kette nach oben bis zur Wurzelklasse `Object`.	**MRO (Method Resolution Order)** mittels C3-Linearisierung löst Konflikte auf.
Zugriffsschutz	Durch Compiler erzwungene Schlüsselwörter: `public`, `protected`, `private`.	Keine echten Sperren; Steuerung über Namenskonventionen (`_intern`, `__mangled`).
Polymorphismus	Basiert fast immer auf gemeinsamer Basisklasse oder gemeinsamen Interfaces.	**Duck Typing**: Entscheidend ist das Vorhandensein der Methode, nicht der Stammbaum.
Funktion von `super()`	Statischer Verweis auf die direkte Elternklasse im Quellcode.	Dynamische Delegation an die *nächste Klasse innerhalb der MRO-Kette*.

DEEP DIVE Die 5 Python-Besonderheiten im Detail

- **1. Echte Mehrfachvererbung (Multiple Inheritance):** Eine Klasse kann gleichzeitig von mehreren Elternklassen erben (z. B. `class Amphibie(Landtier, Wassertier):`). Das ist ideal für sogenannte *Mixin-Klassen*, die kleine Zusatzfunktionen bündeln.
- **2. Method Resolution Order (MRO):** Wenn zwei Elternklassen eine gleichnamige Methode besitzen (das sogenannte *Diamond-Problem*), bestimmt Python über die MRO exakt, welche Klasse Vorrang hat (von links nach rechts, ohne Duplikate). Abrufbar ist diese Liste jederzeit über `Klassenname.mro()` oder `Klassenname.__mro__`.
- **3. Die Dynamik von `super()`:** In Python ruft `super()` nicht starr die Elternklasse auf, sondern folgt der MRO des aktuellen Objekts. Dadurch wird bei Mehrfachvererbung verhindert, dass die oberste Basisklasse versehentlich mehrfach initialisiert wird.
- **4. Namenskonventionen statt `private` / `protected`:** In Python sind alle Attribute öffentlich (`public`). Ein einfacher Unterstrich (`self._wert`) signalisiert anderen Entwicklern: „Bitte nur intern oder in Kindklassen nutzen." Ein doppelter Unterstrich (`self.__wert`) aktiviert das *Name Mangling*: Python benennt das Attribut intern in `_Klassenname__wert` um, um versehentliches Überschreiben in Kindklassen zu verhindern.
- **5. Duck Typing statt Vererbungszwang:** Nach dem Leitspruch *„Wenn es läuft wie eine Ente und quakt wie eine Ente, dann ist es eine Ente"* prüft Python selten den Klassentyp. Eine Funktion, die `objekt.mach_geraeusch()` aufruft, funktioniert auch mit einer Klasse `Auto`, selbst wenn diese gar nicht von `Tier` erbt.